

Secure Storage in Edge-Cloud MPU-based

Bagranov Gleb 560671

As the project for System Security & Web Programming

Legenda

1. Executive Summary.....	3
2. System Architecture	3
2.1 User Flow	3
2.2 Folder Structure	4
3. Security Layers.....	5
3.1 Encryption Key Lifecycle.....	6
4. Cryptographic Implementation	6
4.1 File Encryption (AES-256-CTR)	6
4.2 Filename Encryption	7
4.3 Avatar Encryption	7
4.4 Password Storage	7
5. Screenshots	8
5.1 Login Page	8
5.2 Registration Page	8
5.3 Main Drive Dashboard	9
5.4 Profile Page	9
5.5 Admin Dashboard	10
6. Conclusion.....	10

1. Executive Summary

CloudSim is a web application that simulates a personal encrypted cloud storage service. The application allows users to register an account, upload files to a private storage space, and download them at any time — with every file encrypted on the server before being saved to disk.

The project covers the complete web development stack: a PHP backend, a MySQL database, server-side encryption using the OpenSSL library, and a modern frontend written in HTML, CSS, and JavaScript. The security model is based on the principle that the server should never store encryption keys — each user's key is derived from their password at login and held only for the duration of the session.

The result is a functional, full-stack application that demonstrates both secure web development practices and a practical understanding of cryptographic principles applied to a real-world scenario.

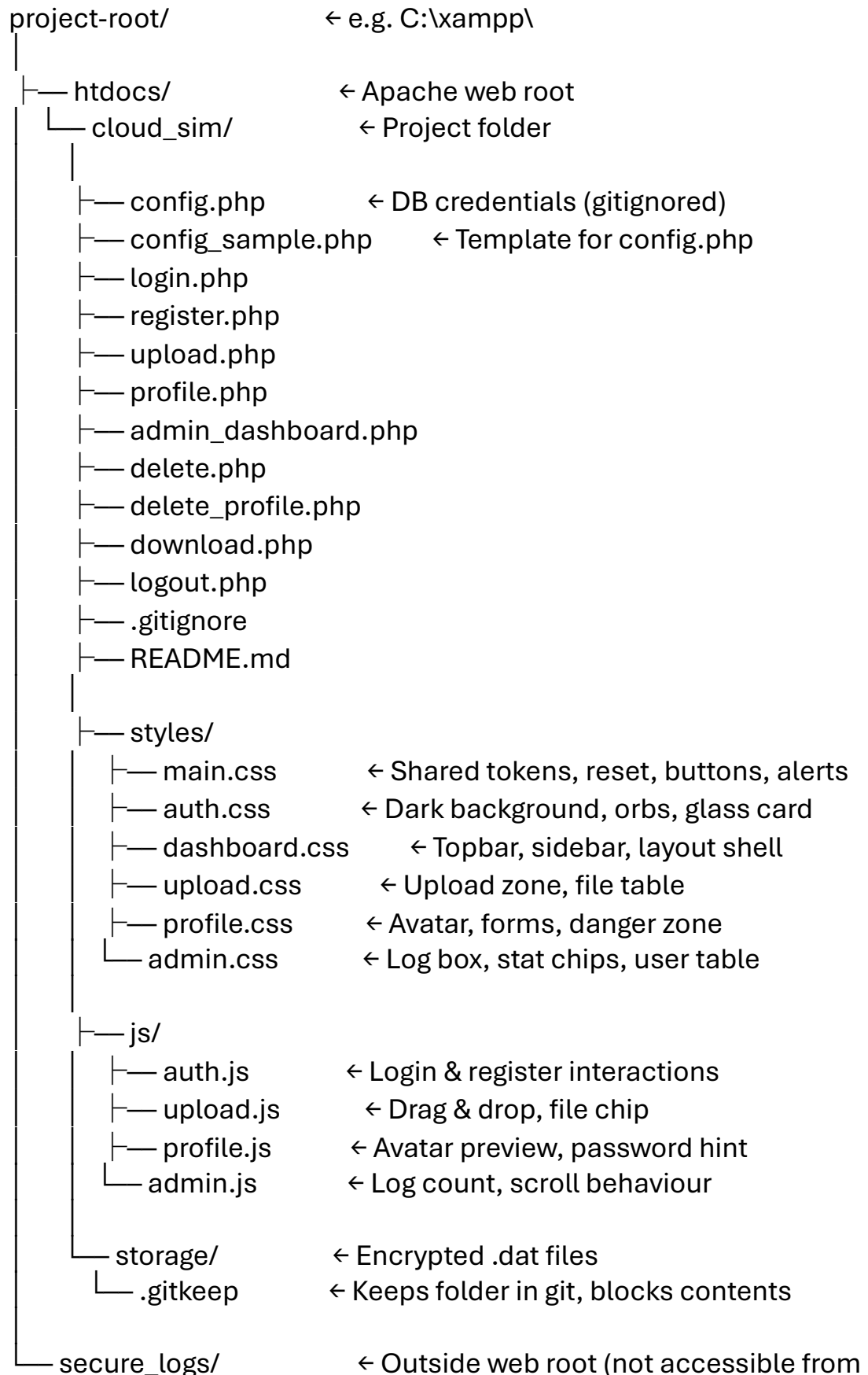
2. System Architecture

2.1 User Flow

The standard user journey through the system follows this sequence:

- User registers with a username, password, and email address
- On login, PBKDF2 derives an encryption key from the password and stores it in the session
- User uploads a file — it is encrypted with AES-256-CTR and saved to disk as a .dat file
- The original filename is also encrypted separately and stored in the database
- On download, the file is decrypted in memory and streamed to the browser with the original name restored
- On logout, the session is destroyed and the encryption key is discarded

2.2 Folder Structure



3. Security Layers

Security Layer	Attack Prevented	How It Works
AES-256-CTR Encryption	Data theft from server	Every uploaded file is encrypted before being written to the disk. The raw .dat files are unreadable without the key.
PBKDF2 Key Derivation	Brute-force on stolen DB	Encryption key is derived from the user's password using 200,000 iterations of SHA-256. The key is never stored anywhere.
CSRF Tokens	Cross-site request forgery	Every form contains a unique session token. POST requests without a valid matching token are rejected with HTTP 403.
Prepared Statements	SQL injection	All database queries use parameterized prepared statements. User input is never concatenated into SQL strings.
MIME + Extension Validation	Malicious file upload	finfo checks the real MIME type from file bytes. A banned extension list blocks PHP, executable, and script files.
Rate Limiting	Brute-force & DoS	Each IP is limited to 5 requests per 10 seconds. Excess requests receive HTTP 429. Timestamps stored outside the web root.
Stealth 404 on Admin Panel	Admin panel discovery	Non-admin users who request admin_dashboard.php receive a fake HTTP 404 Not Found response. The page appears not to exist.

3.1 Encryption Key Lifecycle

The lifecycle of the encryption key is the most critical security design in the project. It works as follows:

- At registration: a unique random `enc_salt` is generated and stored in the users table
- At login: the key is computed as `PBKDF2(password, enc_salt, 200000 iterations, SHA-256)` and stored only in the PHP session
- During upload: the key encrypts the file and the filename before saving
- During download: the key decrypts the file in memory and it is streamed to the browser
- At logout: the session is destroyed and the key ceases to exist on the server
- On password change: a new salt and key are generated, and all stored files are re-encrypted under the new key in the same request

4. Cryptographic Implementation

4.1 File Encryption (AES-256-CTR)

AES-256 in CTR (Counter) mode was chosen for file encryption for two reasons: it does not require padding so the ciphertext is the same length as the plaintext, and it allows random access decryption which is suitable for streaming. The encryption process for each file is:

- Generate a random 16-byte Initialization Vector (IV) using `random_bytes()`
- Read the raw file bytes from the PHP temporary upload buffer
- Encrypt using `openssl_encrypt()` with `OPENSSL_RAW_DATA` flag
- Prepend the IV to the ciphertext and write the result to disk as a `.dat` file
- The IV is not secret — it just ensures the same file encrypted twice produces different ciphertext

4.2 Filename Encryption

Original filenames are encrypted separately from file contents. A dedicated independent IV is generated for each filename — this prevents IV reuse under the same key, which is a known cryptographic weakness. The stored value in the database is: `bin2hex(name_iv)` concatenated with `bin2hex(encrypted_name)`.

4.3 Avatar Encryption

Profile pictures are stored as AES-256-CTR encrypted binary blobs directly in the MySQL database (MEDIUMBLOB column). The same key derivation and encryption scheme is applied as for regular files. The MIME type is stored alongside the blob so the browser knows how to render the image after decryption.

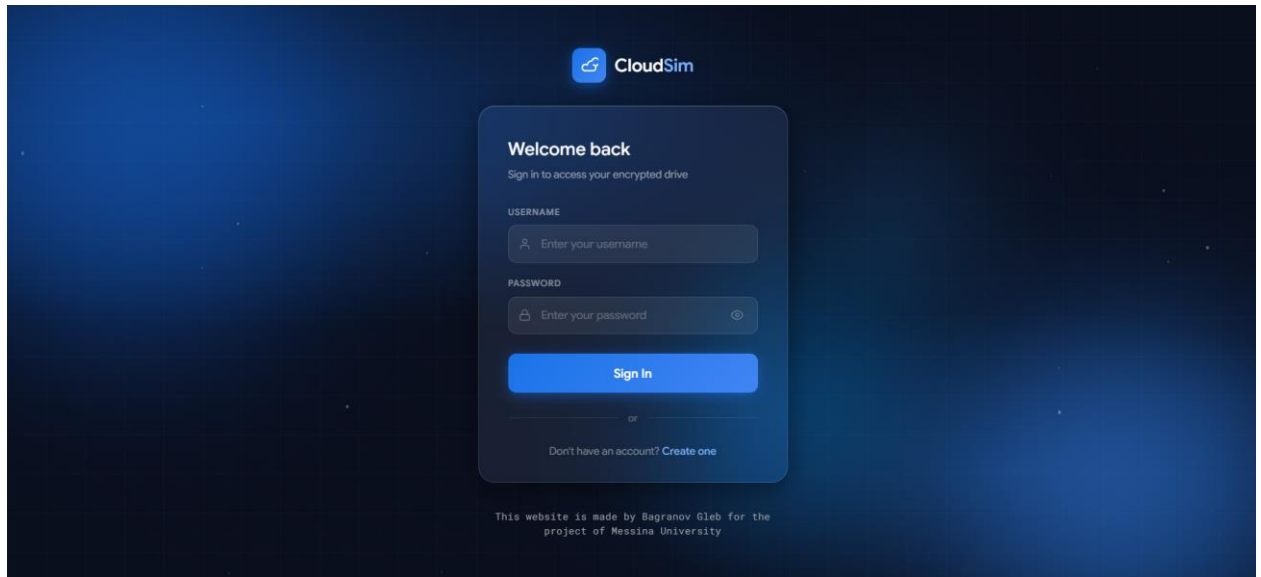
4.4 Password Storage

User passwords are never stored in plaintext or as reversible hashes. PHP's `password_hash()` function with the `PASSWORD_DEFAULT` algorithm (bcrypt) is used at registration. At login, `password_verify()` compares the submitted password against the stored hash in constant time, preventing timing attacks.

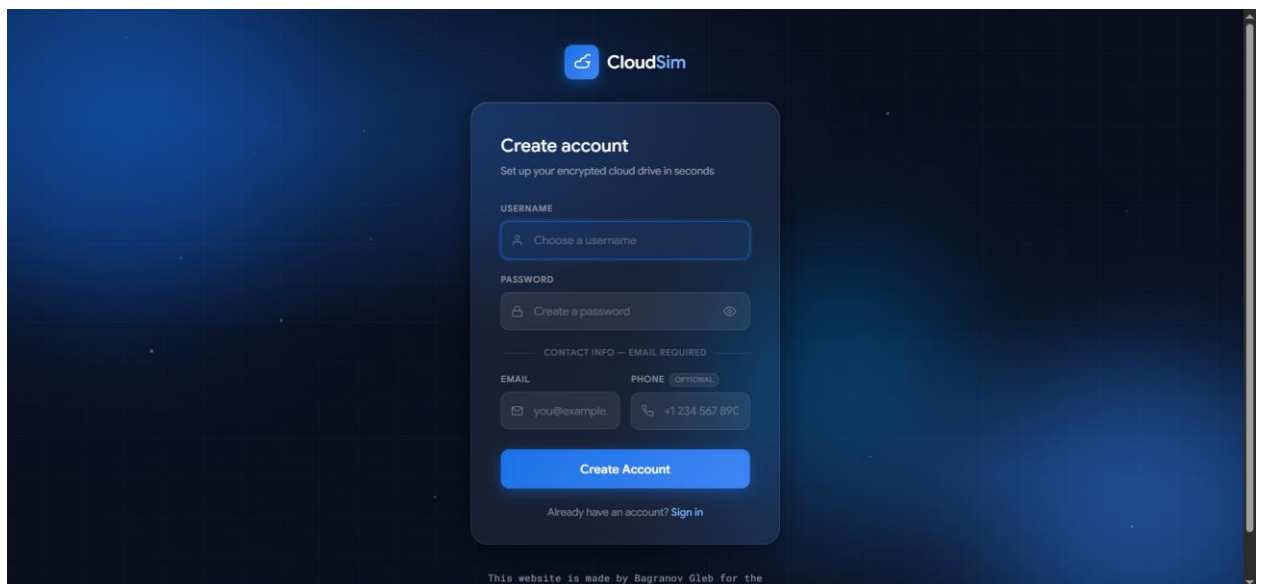
5. Screenshots

The following screenshots show the pages of the CloudSim application.

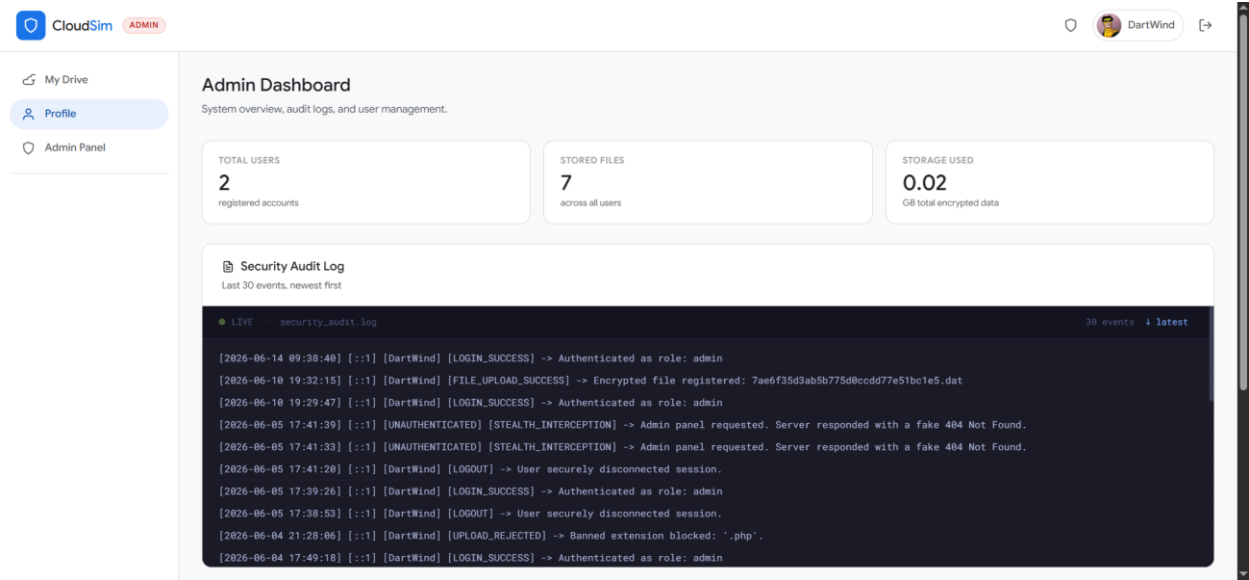
5.1 Login Page



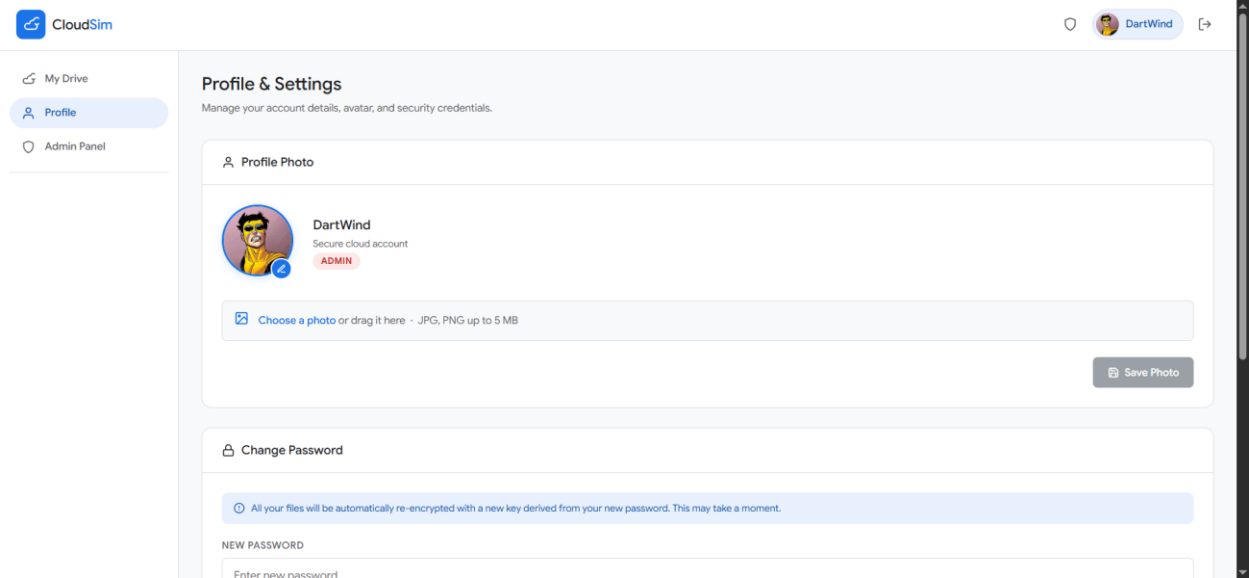
5.2 Registration Page



5.3 Main Drive Dashboard



5.4 Profile Page



5.5 Admin Dashboard

The screenshot displays the CloudSim Admin Dashboard. The top navigation bar includes the CloudSim logo, a user profile icon labeled 'DartWind', and a search icon. The left sidebar contains links for 'My Drive', 'Profile', and 'Admin Panel'. The main content area is titled 'Admin Dashboard' and includes a subtitle 'System overview, audit logs, and user management.' Below this, there are three summary cards: 'TOTAL USERS' with a value of 2 (registered accounts), 'STORED FILES' with a value of 7 (across all users), and 'STORAGE USED' with a value of 0.02 GB total encrypted data. A 'Security Audit Log' section follows, showing the last 30 events. The log is displayed in a dark-themed terminal window with the following entries:

```
[2026-06-14 09:38:40] [::1] [DartWind] [LOGIN_SUCCESS] -> Authenticated as role: admin
[2026-06-10 19:32:15] [::1] [DartWind] [FILE_UPLOAD_SUCCESS] -> Encrypted file registered: 7ae6f35d3ab5b775d0ccdd77e51bc1e5.dat
[2026-06-10 19:29:47] [::1] [DartWind] [LOGIN_SUCCESS] -> Authenticated as role: admin
[2026-06-05 17:41:39] [::1] [UNAUTHENTICATED] [STEALTH_INTERCEPTION] -> Admin panel requested. Server responded with a fake 404 Not Found.
[2026-06-05 17:41:33] [::1] [UNAUTHENTICATED] [STEALTH_INTERCEPTION] -> Admin panel requested. Server responded with a fake 404 Not Found.
[2026-06-05 17:41:20] [::1] [DartWind] [LOGOUT] -> User securely disconnected session.
[2026-06-05 17:39:26] [::1] [DartWind] [LOGIN_SUCCESS] -> Authenticated as role: admin
[2026-06-05 17:38:53] [::1] [DartWind] [LOGOUT] -> User securely disconnected session.
[2026-06-04 21:28:06] [::1] [DartWind] [UPLOAD_REJECTED] -> Banned extension blocked: '.php'.
[2026-06-04 17:49:18] [::1] [DartWind] [LOGIN_SUCCESS] -> Authenticated as role: admin
```

6. Conclusion

CloudSim demonstrates that a secure, functional cloud storage service can be built using only standard web technologies — PHP, MySQL, HTML, CSS, and JavaScript — without external frameworks or third-party services.

The central challenge was designing an encryption architecture where the server never stores user keys. PBKDF2 key derivation, per-file IV management, encrypted filenames, and automatic key rotation on password change all work together transparently from the user's perspective.

The result is a full-stack application that applies production-level security principles — layered defences, secure defaults, and a clear threat model — to a practical and well-structured project.